THE DOCTORS LABORATORY

Developer Programming Challenge

Before you start

This is the first step in our process. Getting it out of the way early means the rest of the conversation can focus on the other things that matter.

We are looking for an answer that is well structured, easy to read, and as simple as possible.

- Please spend no more than two or three hours on this. If you run out of time, send us what you have with a note on what you would do next.
- The deliverable is the program itself. We are not looking for a user interface, a web application, a database or a deployment pipeline, so please do not spend hours building one. Instead, use your README to describe briefly what you would build around the program if it were going further: how you would surface it, where the data would live, how you would deploy and run it.
- Approach it as a piece of work you are delivering inside a cross functional team. Include a README with instructions for running it, an explanation of the technical choices you have made, and an overview of how you approached the problem.
- Unit testing matters to us.
- Use whichever language and frameworks you are most comfortable with. We should be able to run your code without any unusual setup steps.
- Put your code in a public source repository and send us the URL. Please include your commit history.

We assume you will use AI tooling and it will not count against you. Tell us in your README how you used it and where you chose not to.

We will talk through your submission with you at the next stage.

The brief is all yours to interpret. Where it leaves something open, make your own call and note the assumption in your README. We are not expecting any questions before you submit.

Problem: Martian Robots

The surface of Mars can be modelled by a rectangular grid around which robots are able to move according to instructions provided from Earth. You are to write a program that determines each sequence of robot positions and reports the final position of the robot.

A robot position consists of a grid coordinate (a pair of integers: x-coordinate followed by y-coordinate) and an orientation (N, S, E, W for north, south, east, and west).

A robot instruction is a string of the letters “L”, “R”, and “F” which represent, respectively, the instructions:

- **Left**: the robot turns left 90 degrees and remains on the current grid point.
- **Right**: the robot turns right 90 degrees and remains on the current grid point.
- **Forward**: the robot moves forward one grid point in the direction of the current orientation and maintains the same orientation.

The direction North corresponds to the direction from grid point (x, y) to grid point (x, y+1).

There is also a possibility that additional command types may be required in the future and provision should be made for this.

Since the grid is rectangular and bounded (…yes Mars is a strange planet), a robot that moves “off” an edge of the grid is lost forever. However, lost robots leave a robot “scent” that prohibits future robots from dropping off the world at the same grid point. The scent is left at the last grid position the robot occupied before disappearing over the edge. An instruction to move “off” the world from a grid point from which a robot has been previously lost is simply ignored by the current robot.

The Input

The first line of input is the upper-right coordinates of the rectangular world, the lower-left coordinates are assumed to be 0, 0.

The remaining input consists of a sequence of robot positions and instructions (two lines per robot). A position consists of two integers specifying the initial coordinates of the robot and an orientation (N, S, E, W), all separated by whitespace on one line. A robot instruction is a string of the letters “L”, “R”, and “F” on one line.

Each robot is processed sequentially, i.e., finishes executing the robot instructions before the next robot begins execution.

The maximum value for any coordinate is 50.

All instruction strings will be less than 100 characters in length.

The Output

For each robot position/instruction in the input, the output should indicate the final grid position and orientation of the robot. If a robot falls off the edge of the grid the word “LOST” should be printed after the position and orientation.

SAMPLE INPUT	SAMPLE OUTPUT
5 3 1 1 E RFRFRFRF 3 2 N FRRFLLFFRRFLL 0 3 W LLFFFLFLFL	1 1 E 3 3 N LOST 2 3 S